



FLOORPLANMASTER: BLENDER ADD-ON PRO PARAMETRICKÉ MODELOVÁNÍ MÍSTNOSTÍ

Oskar Wladař

Bakalářská práce
Fakulta informačních technologií
České vysoké učení technické v Praze
Katedra softwarového inženýrství
Studijní program: Informatika
Specializace: Počítačová Grafika
Vedoucí: Ing. Lukáš Bařinka
April 20, 2026

České vysoké učení technické v Praze

Fakulta informačních technologií

© 2026 Oskar Wladař. Všechna práva vyhrazena.

Tato práce vznikla jako školní dílo na Českém vysokém učení technickém v Praze, Fakultě informačních technologií. Práce je chráněna právními předpisy a mezinárodními úmluvami o právu autorském a právech souvisejících s právem autorským. K jejímu užití, s výjimkou bezúplatných zákonných licencí a nad rámec oprávnění uvedených v Prohlášení, je nezbytný souhlas autora.

Odkaz na tuto práci: Oskar Wladař. *FloorPlanMaster: Blender add-on pro parametrické modelování místností*. Bakalářská práce. České vysoké učení technické v Praze, Fakulta informačních technologií, 2026.

TODO: Poděkování

Prohlášení

TODO

V Prague dne April 20, 2026

Abstrakt

Tvorba architektonických půdorysů ve volně dostupném 3D modelovacím softwaru Blender je v nativní podobě destruktivní proces: každá úprava rozměrů stěny nebo přesun otvoru vyžaduje ruční korekci sousední geometrie, což výrazně narušuje plynulost návrhového procesu. Žádný z dostupných rozšiřovacích modulů pro Blender přitom neposkytuje komplexní nástroj pro přirozené kreslení půdorysu v kombinaci s automatickou detekcí místností a nedestruktivní správou otvorů.

Cílem práce bylo navrhnout a implementovat rozšiřovací modul FloorPlanMaster pro Blender, který umožňuje parametrické a nedestruktivní modelování prostorových dispozic. Na základě analýzy tří cílových skupin (architekti, 3D vizualizátoři a game designéři) a komparativní analýzy existujících nástrojů byly strukturovány funkční a nefunkční požadavky a navržena třívrstvá hybridní architektura kombinující strukturální graf pro topologii stěn a styků, sémantický graf místností pro automatickou detekci uzavřených cyklů a synchronizační vrstva přenášející data do Blender meshe s pojmenovanými atributy čtenými modulem Geometry Nodes.

Výsledkem je funkční add-on implementovaný v Pythonu zahrnující interaktivní nástroj pro kreslení stěn (modální operátor), parametrickou editaci tloušťky a výšky stěn, automatickou detekci místností a jejich ploch a podporu otvorů (dveře, okna) s poziční validací.

Hlavním přínosem práce je přehledný a detailní nástroj, který architektům a vizualizátorům umožňuje celý pracovní postup — od prvního náčrtu dispozice přes parametrické úpravy až po finální mesh — realizovat v jednom prostředí bez nutnosti přepínat mezi specializovanými programy. Addon tím zároveň zaplňuje identifikovanou mezeru v ekosystému Blenderu a přináší workflow srovnatelné s profesionálními architektonickými nástroji přímo do prostředí, které uživatelé již dobře znají.

Klíčová slova Blender add-on, parametrické modelování, půdorys, architektonická vizualizace, Geometry Nodes, grafový datový model, prostorová dispozice, Python API

Abstract

Creating architectural floor plans in the freely available 3D modelling software Blender is, in its native form, a destructive process: every modification of wall dimensions or repositioning of an opening requires manual correction of the surrounding geometry, significantly disrupting the fluidity of the design workflow. None of the available extensions for Blender provides a comprehensive tool for natural floor plan sketching combined with automatic room detection and non-destructive opening management.

The aim of this thesis was to design and implement the FloorPlanMaster extension for Blender, enabling parametric and non-destructive modelling of spatial floor plans. Based on the analysis of three target user groups (architects, 3D visualizers, and game designers) and a comparative analysis of existing tools, functional and non-functional requirements were elicited and a three-layer hybrid architecture was designed, combining a structural graph for wall and junction topology, a semantic room graph for automatic detection of closed cycles, and a synchronization layer transferring data into the Blender mesh with named attributes read by the Geometry Nodes module.

The result is a functional add-on implemented in Python, encompassing an interactive wall drawing tool based on a modal operator, parametric editing of wall thickness and height, automatic room detection with area computation, and support for openings (doors and windows) with positional validation.

The primary contribution of the thesis is a clear and detailed tool that enables architects and visualizers to carry out the entire workflow — from the initial floor plan sketch through parametric editing to the final mesh — within a single environment, without the need to switch between specialized applications. In doing so, the add-on fills an identified gap in the Blender ecosystem and brings a workflow comparable to professional architectural tools directly into an environment users already know.

Keywords Blender add-on, parametric modeling, floor plan, architectural visualization, Geometry Nodes, graph data model, spatial layout, Python API

Obsah

Úvod	1
1 Analýza	2
1.1 Doménová analýza	2
1.1.1 Parametrické modelování a prostorová dispozice	3
1.1.2 Analýza existujících řešení	3
1.2 Cílové skupiny	6
1.2.1 Uživatelské skupiny	6
1.2.2 Persony	7
1.3 Vstupy a výstupy	8
1.3.1 Vstupy	9
1.3.2 Výstupy	9
1.4 Scénáře použití	9
1.4.1 UC 1.1: Hmotová studie na základě stavebního programu	10
1.4.2 UC 1.2: Kreslení dispozice tužkou	10
1.4.3 UC 1.3: Kontrola rozměrů vůči normovým minimům	10
1.4.4 UC 2.1: Obkreslení dodaného 2D půdorysu	10
1.4.5 UC 2.2: Příprava modelu pro renderovací pipeline	11
1.4.6 UC 2.3: Rychlá editace vlastností prvků přes kontextovou nabídku	11
1.4.7 UC 3.1: Rychlý level blackout	11
1.4.8 UC 3.2: Finalizace a export herní úrovně	11
1.4.9 UC 3.3: Interaktivní úprava rozložení místností	12
1.5 Analýza požadavků	12
1.5.1 Funkční požadavky	12
1.5.2 Nefunkční požadavky	14
1.5.3 Prioritizace požadavků	15
1.6 Technická analýza	17
1.6.1 Architektura Blenderu	17
1.6.2 Interaktivní kreslení a interakce ve viewportu	17
1.6.3 Reprezentace geometrie	19
1.6.4 Datový model	20
1.6.5 Tvorba otvorů pro okna a dveře	22
1.6.6 Ukládání dat a správa metadat	23
1.6.7 Uživatelské rozhraní	25
1.6.8 Finalizační nástroj	27
2 Návrh	28
3 Implementace	28
4 Testování	28

Závěr	29
Bibliografie	30
Contents of the attachment	31

Seznam výpisů kódu

Seznam tabulek

Tabulka 1	Mapování funkčních balíčků na scénáře použití (● = relevantní)	15
Tabulka 2	Vážená prioritizace požadavků podle cílových skupin	16
Tabulka 3	Návratové hodnoty modálního operátoru	18
Tabulka 4	Srovnání BMesh a Geometry Nodes pro generování 3D stěn	20
Tabulka 5	Srovnání metod pro tvorbu otvorů ve stěnách	23
Tabulka 6	Přístupy k perzistenci grafových dat v Blenderu	25

Seznam obrázků

Seznam zkratek

<i>API</i>	– Application Programming Interface	4
<i>BIM</i>	– Building Information Modeling	2
<i>BLF</i>	– Blender Font Library	13
<i>CAD</i>	– Computer-Aided Design	2
<i>DNA</i>	– Data Architecture (Blender's internal C-structure layer)	17
<i>DWG</i>	– Drawing — AutoCAD native binary format	9
<i>DXF</i>	– Drawing Exchange Format	9
<i>FBX</i>	– Filmbox — Autodesk proprietary 3D interchange format	9
<i>GIL</i>	– Global Interpreter Lock	20
<i>GN</i>	– Geometry Nodes	19
<i>GPU</i>	– Graphics Processing Unit	13
<i>HUD</i>	– Heads-Up Display	26
<i>IFC</i>	– Industry Foundation Classes	4
<i>ISO</i>	– International Organization for Standardization	4
<i>JSON</i>	– JavaScript Object Notation	25
<i>MVC</i>	– Model-View-Controller	17
<i>OBJ</i>	– Wavefront OBJ — open 3D geometry file format	9
<i>PDF</i>	– Portable Document Format	6
<i>RMB</i>	– Right Mouse Button	26
<i>RNA</i>	– Runtime Notification Architecture (Blender's Python-accessible reflection layer)	17
<i>SVG</i>	– Scalable Vector Graphics	4
<i>UI</i>	– User Interface	14
<i>UV</i>	– UV coordinates — 2D texture mapping coordinates	11
<i>UX</i>	– User Experience	16

Úvod

Bakalářská práce se zabývá návrhem a implementací rozšiřovacího modulu **FloorPlanMaster** pro 3D software Blender. Cílem je nástroj, který do Blenderu přináší parametrické, nedestruktivní modelování prostorových dispozic — umožňuje interaktivně kreslit půdorysy, definovat místnosti a upravovat rozměry jednotlivých prvků bez nutnosti ručně přestavovat geometrii.

Blender je výkonný, volně dostupný 3D software s aktivní komunitou vývojářů rozšiřovacích modulů. Přestože dnes slouží i pro komplexní architektonické projekty, pro tvorbu půdorysů a prostorových dispozic mu chybí specializované nástroje. Existující rozšíření tento nedostatek adresují jen částečně — obvykle buď postrádají skutečně nedestruktivní přístup, nebo jsou příliš složité pro ranou fázi konceptuálního návrhu. Výsledkem je mezera, kterou FloorPlanMaster cílí zaplnit. Addon je určen zejména architektům a vizualizátorům, kteří potřebují rychle skicovat dispozice v prostředí, které již znají, aniž by přecházeli do specializovaného CAD nebo BIM softwaru.

Řešení je navrženo jako vícevrstvá architektura, která odděluje logiku půdorysu od jeho vizualizace. Datový model půdorysu je spravován čistě v Pythonu a komunikuje s Blenderem jednosměrně — změna parametru se okamžitě promítne do 3D zobrazení, aniž by bylo nutné jakkoliv zasahovat do výsledné geometrie. Addon je implementován jako rozšiřující modul pro Blender napsaný v Pythonu a nevyžaduje žádný externí software.

Struktura práce postupuje od obecného kontextu k technickým detailům. Analytická kapitola vymezuje problémovou doménu, definuje cílové skupiny uživatelů a odvozuje požadavky na addon. Kapitola návrhu překládá tyto požadavky do softwarové architektury a návrhu funkcí. Následují kapitoly věnované implementaci, testování a závěrečnému zhodnocení dosažených výsledků.

Kapitola 1

Analýza

Blender je všestranný, ale pro architektonické skicování nevhodně vybavený nástroj — každá změna půdorysu se mění v destruktivní opravu vrcholů a přepočítávání otvorů, což narušuje plynulost návrhového procesu. Aby bylo možné FloorPlanMaster navrhnout správně, je nutné nejprve porozumět tomu, proč stávající řešení nestačí, kdo addon skutečně bude používat a co od něj v konkrétních situacích čeká. Tato analýza tedy postupně buduje obraz od problémové domény přes uživatele a jejich scénáře až ke strukturovaným požadavkům a výběru technologií, na nichž addon stojí.

1.1 Doménová analýza

Architektonická dispozice vzniká iterativně: místnosti se posouvají, chodby se zužují, stěny mění tloušťku — a každá z těchto změn by měla být otázkou sekund, ne minut ručního přepočítávání geometrie. Cílem doménové analýzy je pochopit, proč Blender jako obecný nástroj tento požadavek nativně nenaplnuje a jaký typ řešení by mezeru zaplnil. Nejprve je proto potřeba vymezit, čím se parametrické modelování liší od klasické polygonální práce, a poté zmapovat, jak se s tímto problémem vypořádala stávající řešení — jak uvnitř prostředí Blenderu, tak mimo něj.

Blender [1] je primárně 3D polygonální modelovací nástroj, nikoliv specializovaný CAD nebo BIM software. Nabízí obrovskou flexibilitu, pro specifické potřeby architektonického navrhování však postrádá nativní nástroje, což vede k neefektivním a zdoluhavým pracovním postupům. Tvorba půdorysů a 3D dispozic budov je v čistém Blenderu zdoluhavá a destruktivní — každá změna vyžaduje manuální opravu okolní geometrie, posouvání vrcholů a přepočítávání otvorů.

Cílem projektu je tedy vytvoření rozšiřovacího modulu pro Blender zaměřeného na parametrické modelování prostorových dispozic. Uživatel bude moci prostorové dispozice intuitivně vytvářet a nedestruktivně upravovat pomocí parametrů; primárním cílem je urychlit ranou fázi architektonického návrhu.

1.1.1 Parametrické modelování a prostorová dispozice

Parametrické modelování je způsob vytváření 3D modelů, kde tvar objektu není definován pevně, ale pomocí proměnných parametrů (číselných hodnot) a geometrických vztahů. Tvůrce přímo kontroluje vstupy a algoritmickou logiku závislostí: čtverec lze například nadefinovat parametrem šířky, parametrem délky a pravidlem kolmosti stran, přičemž pozdější změna parametrů způsobí automatické překreslení tvaru bez nutnosti manuálního zásahu.

Oproti klasickému polygonálnímu modelování, kde se pracuje s body, hranami a plochami jejich přímou manipulací, parametrické modelování pracuje s čísly, funkcemi a historií kroků. Polygonální modelování je primárně vhodné pro hry, filmy a animace; parametrické modelování převládá v architektuře, strojírenství a produktovém designu.

Rozlišují se dvě hlavní paradigmatu [2], [3]. *Historické* modelování, typické pro CAD a BIM software, si pamatuje časovou osu úprav: software zaznamenává sekvenci kroků a umožňuje vrátit se k dřívějšímu kroku a automaticky přepočítat všechny závislé operace. Toto paradigma je standardem v průmyslovém CAD (SolidWorks) i architektonickém BIM (Revit). *Algoritmické* modelování, označované také jako vizuální skriptování, popisuje geometrii pomocí uzlových grafů a je analogické Geometry Nodes v Blenderu.

Parametrické modelování v architektuře přináší posun od tradičního reprezentativního kreslení k algoritmickému přístupu. Tradiční CAD systémy se spoléhají na explicitní definici geometrie pomocí statických bodů a úseček. Parametrický přístup zavádí systém vzájemně propojených proměnných, matematických omezení a deduktivních pravidel, která dynamicky generují a aktualizují výslednou formu. Úprava jediného parametru — například celkové výšky podlaží nebo tloušťky nosné stěny — automaticky a kaskádovitě modifikuje všechny závislé entity.

Prostorová dispozice je logické a funkční uspořádání trojrozměrného objektu do smysluplných celků (místností a zón) s definovanými vztahy mezi nimi. Tento pojem vymezuje konkrétní předmět návrhu, se kterým addon pracuje.

1.1.2 Analýza existujících řešení

Před návrhem nového nástroje je nutné pochopit, co již existuje a kde existující řešení selhávají — jinak hrozí, že FloorPlanMaster jen zopakuje jejich slabiny. Tato sekce proto hodnotí nejvýraznější architektonické addony přímo pro Blender a porovnává je s nástroji mimo jeho ekosystém, aby bylo jasné, jaká mezera zůstala nezaplněna.

1.1.2.1 Architektonické rozšiřující moduly pro Blender

Blender za posledních deset let prošel dramatickou evolucí. Původně vnímaný jako nástroj pro organické modelování, animace a vizuální efekty byl díky open-source modelu a robustnímu Python [API](#) transformován v platformu schopnou realizovat komplexní architektonické projekty. Vzniklo tak několik specializovaných addonů.

Archimesh [\[4\]](#) je základním kamenem architektonických nástrojů pro Blender. Vytvořil ho Antonio Vazquez s cílem automatizovat tvorbu interiérových a exteriérových prvků, která by jinak zabrala hodně času manuálním modelováním. Díky stabilitě a užitečnosti byl dlouhodobě integrován přímo do oficiální distribuce Blenderu jako komunitní doplněk a je primárně určen pro rychlé skicování prostor a interiérový design.

Pro tvorbu místností nabízí Archimesh dva přístupy: definování počtu stěn a jejich rozměrů, nebo využití nástroje Grease Pencil, kde uživatel v půdorysném pohledu nakreslí hrubé obrysy místností a doplněk tyto tahy automaticky převede na trojrozměrné stěny. Addon dále umožňuje automatické generování podlah a stropů. Co se týče otvorů, podporuje dva typy oken — kolejnicová a panelová — s parametrickými parapety a systémem žaluzií. Součástí je rovněž generátor kuchyňských linek, polic a dalšího interiérového vybavení.

Archipack [\[5\]](#) byl vytvořen jako robustnější a výkonnější alternativa k Archimesh, zaměřená primárně na profesionální architekty a vizualizátory. Autorem je Stephen Leger a addon existuje ve dvou verzích — Community Edition a Pro. Klíčovým rysem je důraz na interaktivní manipulaci: systém Auto-manipulate on select při výběru objektu zobrazí přímo ve 3D viewportu táhla (gizmos) a textové popisky. Parametry prvků jsou spravovány vlastním systémem vlastností, které zůstávají zachovány po celou dobu životnosti projektu — uživatel může kdykoliv vybrat stěnu a změnit její tloušťku nebo výšku a všechny připojené prvky automaticky na tuto změnu reagují. Pro export nabízí Archipack generování řezů a půdorysů ve formátu [SVG](#). Pro verze pak export do formátu [IFC](#).

BonsaiBIM [\[6\]](#) zaujímá zcela odlišnou pozici: na rozdíl od Archimesh a Archipack je navržen jako nativní platforma pro tvorbu a správu informačních modelů budov, fungující na mezinárodním standardu IFC [\[7\]](#) ([ISO 16739](#)). Zatímco Archimesh a Archipack jsou nadstavby nad standardním modelovacím procesem a jejich data jsou spjata s `.blend` souborem, BonsaiBIM umožňuje Blenderu fungovat jako prohlížeč a editor databáze IFC.

1.1.2.2 Architektonické nástroje mimo Blender

Mimo ekosystém Blenderu existují tři dominantní nástroje pro architektonické modelování, která tvoří referenční rámec pro analýzu potřeb cílových skupin.

SketchUp [8] je postaven na principu přímého modelování ploch a jako prioritu klade rychlost a intuitivní transformaci myšlenky do 3D formy. **AutoCAD** [9] představuje standard pro 2D dokumentaci a detailní rýsování — funguje jako digitální rýsovací prkno, nepostradatelné pro technické výkresy. **Revit** [10] je robustní parametrický BIM nástroj, kde každý prvek v modelu je instancí v databázi s definovanými funkčními vztahy; změna v jednom zobrazení se automaticky promítne do všech výkresů a výkazů.

1.1.2.3 Identifikované mezery a příležitosti

Přehled existujících řešení odhaluje opakující se nedostatky, které představují hlavní příležitosti pro budoucí návrh modulu FloorPlanMaster.

Chybějící nástroj pro přirozené kreslení půdorysů. Ani Archimesh, ani Archipack nenabízejí plnohodnotný nástroj pro skicování dispozice klikáním bodů přímo ve viewportu způsobem srovnatelným se SketchUpem nebo AutoCADem. Archimesh sice umí interpretovat tahy Grease Pencil jako stěny a Archipack vygeneruje stěny z křivky, ale ani v jednom případě nevzniká dojem přirozené „tužky v půdorysu,“. FloorPlanMaster proto může zavést interaktivní Pencil Tool jako primární vstupní metodu — modální operátor, který by sledoval kurzor a při každém kliknutí okamžitě vytvořil stěnu.

Nespolehlivá správa otvorů. Systém automatických otvorů v modulu Archimesh má dokumentované slabiny u složitějších stěn a při použití solveru Exact. Otvory v Archipacku sice fungují robustně, ale vložení okna nebo dveří často vede k nepřehlednému zanořování do složitých panelů nastavení. Zde se nabízí prostor pro záměrné zjednodušení rozhraní: otvor by se dal přidávat pouhým výběrem stěny a zadáním tří základních hodnot (šířka, výška, výška parapetu), zatímco o geometrickou konzistenci by se staral modifikátor založený na Geometry Nodes (Mesh Boolean) propojený datovou vazbou.

Chybějící přehled o místnostech a plochách. Žádný z analyzovaných addonů pro Blender nedetekuje uzavřené cykly stěn jako místnosti automaticky a nezobrazuje jejich plochu jako primární informaci. Analytické výměry jsou v Archipacku i Archimesh skryté v technickém rozhraní, nebo zcela chybí. Přitom právě integrace detekce místností a zobrazení ploch přímo do hlavního přehledu v N-panelu by představovala klíčový výstup jak pro architekta, tak pro game designéra při ověřování parametrů návrhu.

Izolace externích nástrojů mimo ekosystém Blenderu. SketchUp, AutoCAD a Revit sice ve svém oboru vynikají, ale postrádají integraci do prostředí Blenderu — přechod z konceptuálního modelu ve SketchUpu do plnohodnotné renderovací scény v Blenderu vyžaduje export, čištění topologie a vede ke ztrátě parametrických vlastností. Cílem FloorPlanMasteru je tuto bariéru eliminovat a zajistit, aby celý životní cyklus — od prvního načrtnutí

dispozice přes parametrické úpravy až po finalizovanou síť (mesh) pro rendering — probíhal přímo v jedné scéně v rámci Blenderu.

1.2 Cílové skupiny

Parametrické modelování půdorysů v Blenderu může urychlit práci architektovi skicujícím varianty pro klienta, vizualizátorce převádějící 2D výkresy z [PDF](#) do 3D prostoru, i game designérovi, který po sérii testování potřebuje narychlo rozšířit herní chodbu. Každý z nich však od addonu očekává něco trochu jiného. Cílem této sekce je tyto skupiny přesně vymezit, pojmenovat jejich konkrétní frustrace s nativním Blenderem a ztělesnit je v uživatelských personách. Ty pak poslouží jako měřítko pro hodnocení každého budoucího návrhového rozhodnutí.

1.2.1 Uživatelské skupiny

Společným problémem všech tří skupin je frustrace z toho, že Blender nedokáže udržet krok s rychlostí jejich myšlení: potřeba změnit dispozici často přichází ve chvíli, kdy je geometrie již pevně spojená v jeden celek a její úprava je zdoluhavá. Každá skupina však naráží na specifické problémy a upřednostňuje jiné vlastnosti addonu, proto je nutné popsat je zvlášť.

1.2.1.1 1. Architekti ve fázi konceptuálního navrhování

Architekti pracující na konceptuálním návrhu tvoří primární cílovou skupinu addonu. Jedná se o profesionály nebo studenty tvořící úvodní hmotové a prostorové studie. V této fázi se nezdržují mikrodaily (jako je typ kování u dveří), ale zaměřují se na celkové proporce, návaznosti místností a celkové uspořádání půdorysu.

Vyžadují rychlou iteraci návrhu, přesné zadávání číselných hodnot (délky, šířky, výšky) a nedestruktivní úpravy — tedy možnost kdykoliv změnit parametry místnosti nebo plynule posouvat okna a dveře podél stěn. Modelování těchto prvků pomocí standardních nástrojů Blenderu pro ně představuje destruktivní proces: každá změna vyžaduje manuální opravu okolní geometrie, posouvání vrcholů a složité přepočítávání otvorů. To odvádí jejich pozornost od samotného navrhování a narušuje plynulost tvůrčí práce.

1.2.1.2 2. 3D umělci a vizualizátoři

Vizualizátoři potřebují co nejrychleji postavit základní geometrii místností, aby se mohli naplno věnovat tomu hlavnímu — materiálům, nasvícení a samotnému renderingu. Jde o tvůrce zaměřené primárně na estetiku. Hrubá

stavba pro ně představuje pouhé „plátno“, do kterého následně vkládají detailní modely nábytku a vybavení.

Kromě rychlosti je pro ně klíčová také čistá topologie výsledné geometrie, na které budou bezchybně fungovat textury a další renderovací modifikátory. Ruční modelování stěn a vyřezávání oken pomocí nativních Boolean modifikátorů totiž často vytváří nevzhlednou topologii a její ruční čištění je pro tyto umělce nepříjemnou a vysoce neefektivní rutinou.

1.2.1.3 3. Game a level designéri

Game a level designéri využívají addon k rychlé tvorbě a iteraci herních blockoutů — hrubých modelů úrovní sloužících k testování hratelnosti, průchodnosti a pohybu hráče či kamery.

Požadují především modulární přístup k tvorbě prostoru, schopnost okamžitě měnit proporce na základě zpětné vazby z herního testování a bezproblémový export hotových tvarů do enginů jako Unreal Engine nebo Unity. Nativní modelovací nástroje Blenderu nejsou pro rychlý level design optimalizovány, a tak je ruční upravování rozměrů místností nebo přesouvání otvorů během prototypování příliš těžkopádné.

1.2.2 Persony

Abstraktní popis cílových skupin neukáže, jak přesně se zmíněné frustrace projevují v praxi — zda architektka brzdí přepočítávání návazných stěn, nebo spíše chybějící přehled o podlahové ploše; zda vizualizátorku zdržuje samotné modelování, nebo až následné čištění topologie po ořezu. Pro každou skupinu je proto definována jedna konkrétní persona. Její profil, cíle a frustrace slouží jako referenční bod při rozhodování o podobě a funkčnosti addonu.

1.2.2.1 1. Architekt Adam

Adam (32 let) pracuje v menším architektonickém ateliéru a má na starosti úvodní studie a komunikaci s klienty při hledání tvaru a dispozice budovy. Běžně pracuje v profesionálních CAD a BIM programech (AutoCAD, Revit), ale pro rychlé 3D koncepty a objemové studie si oblíbil Blender díky jeho svižnosti a real-time zobrazení. Neumí však programovat a práce s Geometry Nodes je pro něj příliš složitá.

Jeho typickým cílem je během pár hodin vytvořit pro klienta tři různé varianty prostorového uspořádání domu. Primárně ho zajímá hmota, návaznosti místností a základní rozměry. Frustruje ho zdlouhavé extrudování polygonů v nativním Blenderu: když klient požádá o rozšíření obývacího pokoje o metr, Adam musí ručně posouvat vertexy a složitě přepočítávat navazující stěny, přičemž mu navíc chybí okamžitý přehled o rozměrech místností. Od

addonu očekává jednoduché rozhraní, ve kterém zadá rozměry místnosti nebo je naskicuje, a systém při jakékoliv změně automaticky zachová tloušťku zdiva a nerozbije návaznost na další prostory.

1.2.2.2 2. Vizualizátorka Věra

Věra (28 let) je vizualizátorka na volné noze, která se specializuje na tvorbu fotorealistických interiérů pro developery a realitní kanceláře. Blender ovládá na špičkové úrovni — má hluboké znalosti materiálů, nasvícení i renderingu — a profiluje se spíše umělecky než technicky.

Běžně dostává od klienta 2D půdorys v PDF a potřebuje z něj co nejrychleji vytvořit hrubý 3D obraz bytu, aby se mohla věnovat tomu hlavnímu: světlu, texturám a vybavení. Zdlouhavé modelování holých stěn ji zdržuje a odvádí od kreativní práce. Navíc nativní vyřezávání otvorů přes Boolean jí často zaneřádí síť ošklivou topologií, kterou pak musí před renderingem složitě čistit. Očekává proto nástroj podobný tužce, kterým podložený půdorys jednoduše obkreslí, a možnost vkládat otvory na jedno kliknutí s jistotou, že addon udrží topologii čistou.

1.2.2.3 3. Level designer Denis

Denis (25 let) je vývojář v nezávislém herním studiu. Navrhuje herní úrovně a testuje pohyb hráče v prostoru. Primárně pracuje v herních enginech (Unreal Engine, Unity), přičemž Blender využívá k rychlé tvorbě takzvaných blockoutů — hrubé geometrie určené pro okamžité testování hratelnosti.

Jeho úkolem je v krátkém čase vybudovat rozsáhlou herní mapu (například spleť chodeb a místností), vyexportovat ji do enginu a projít si ji s herní postavou. Když při testování zjistí, že je chodba příliš úzká nebo strop příliš nízký, je úprava takové úrovně v čistém Blenderu (pokud je spojená do jednoho souvislého kusu geometrie) neefektivní a zdlouhavá. Od addonu proto vyžaduje robustnost a rychlost iterace: parametrický přístup by mu měl umožnit kliknout na chodbu, v panelu přepsat její šířku ze dvou metrů na tři, a nechat systém automaticky vyřešit zbytek. Nezbytností je pro něj také export, který po přesunu do enginu nerozbije fyzikální kolize.

1.3 Vstupy a výstupy

FloorPlanMaster je navržen tak, aby dokázal reagovat na reálné pracovní situace: architekt často začíná s prázdnou scénou a úvodní myšlenkou, vizualizátorka dostane od klienta 2D půdorys ve formátu PDF a game designér vychází z přibližných rozměrů v herním design dokumentu. Tato sekce jasně

vymezuje, jaké formy vstupních dat addon zpracovává a jaké výstupy následně produkuje, čímž definuje hranice životního cyklu celého modelu.

1.3.1 Vstupy

Addon dokáže zpracovat tři základní kategorie vstupů. První kategorií jsou **2D podklady** — naskenované ruční skice, výkresy v PDF, obrázky půdorysů nebo importované 2D CAD výkresy (**DXF**/**DWG**), které uživatel potřebuje převést do 3D prostoru. Podkladový soubor se typicky umístí na pozadí scény a slouží jako vizuální reference pro následné obkreslování.

Druhou kategorií je **kvantitativní zadání** — přesný seznam požadavků od klienta nebo technické specifikace (např. „obývací pokoj musí mít minimálně 30 m²“ nebo „minimální šířka chodby jsou 2 metry“). Tyto hodnoty lze přímo zadávat do panelu nástroje jako číselné parametry.

Třetí kategorií je **volný návrh** — situace, kdy uživatel nemá žádné přesné podklady, začíná s prázdnou scénou a potřebuje nástroj, který ho nebude omezovat v rychlém a intuitivním skicování úvodních konceptů.

1.3.2 Výstupy

Výstupy z addonu se dělí do tří kategorií. Tou první je **3D hmotový model** — prostorová 3D reprezentace stěn, místností a otvorů, která slouží primárně k vizuální kontrole proporcí, tvorbě hmotových renderů, analýze osvětlení nebo k základní prezentaci klientovi.

Druhou kategorií je **optimalizovaná geometrie pro export**. Jedná se o čistou topologickou síť (mesh) bez chyb nebo překrývajících se stěn, kterou může level designér okamžitě a bez dalších úprav vyexportovat (například do formátu **FBX** nebo **OBJ**) pro použití v herním enginu.

Třetí a poslední kategorií jsou **prostorová a analytická data** — rychlá vizuální zpětná vazba pro uživatele. Zahrnuje automatické výpočty podlahové plochy jednotlivých místností a jasnou indikaci tloušťky stěn přímo v uživatelském rozhraní.

1.4 Scénáře použití

Abstraktní požadavky typu „parametrické úpravy“ či „nedestruktivní workflow“ samy o sobě nedefinují, s jakou odezvou se musí stěna přepočítat po kliknutí myši, ani jak se systém zachová, když uživatel do scény přiloží PDF výkres a začne jej obkreslovat. Tyto situace konkretizují až scénáře použití (Use Cases). Každý scénář sleduje konkrétní personu od jejího prvního kliknutí

až po požadovaný výsledek a jasně odkrývá, které funkce modulu jsou pro daný úkol klíčové.

1.4.1 UC 1.1: Hmotová studie na základě stavebního programu

Architekt zadá do panelu nástroje požadovanou podlahovou plochu a poměr stran pro každou místnost (například 30 m², poměr 1:1,5). Addon automaticky vypočítá potřebné rozměry a vloží pravoúhlou místnost přímo do scény. Uživatel tento postup zopakuje pro všechny místnosti ze stavebního programu. Výsledkem je schematická dispozice, u níž lze v panelu okamžitě ověřit plochu každého prostoru.

1.4.2 UC 1.2: Kreslení dispozice tužkou

Architekt aktivuje kreslicí nástroj a pouhým klikáním bodů přímo ve 3D viewportu načrtne hrubou dispozici. Addon průběžně generuje stěny a při uzavření polygonu automaticky detekuje vzniklé místnosti. Uživatel následně doladí tloušťku a výšku stěn zadáním přesných hodnot v N-panelu.

1.4.3 UC 1.3: Kontrola rozměrů vůči normovým minimům

Architekt má navrženou dispozici a potřebuje ověřit, zda šířky chodeb a rozměry místností splňují minimální normové požadavky. V N-panelu zapne kótovací vrstvu (overlay). Addon prostřednictvím překreslovacího modulu (BLF draw handler) okamžitě zobrazí délky všech stěn a plochy místností přímo ve viewportu. Uživatel tak může vizuálně zkontrolovat kritická místa a případně upravit parametry stěn v panelu.

1.4.4 UC 2.1: Obkreslení dodaného 2D půdorysu

Vizualizátorka si na pozadí scény vloží referenční obrázek s půdorysem. Aktivuje kreslicí nástroj a se zapnutým přichytáváním (snapping) na existující uzly odklikává rohy místností přesně podle podkladu. Addon během kreslení průběžně generuje stěny a při uzavření cyklů automaticky detekuje jednotlivé místnosti. Výšku a tloušťku stěn následně hromadně nebo individuálně nastaví v N-panelu.

1.4.5 UC 2.2: Příprava modelu pro renderovací pipeline

Vizualizátorka vybere na existujícím půdorysu konkrétní stěnu a v N-panelu k ní přidá otvor zadáním přesných rozměrů (například 1500×1250 mm). Addon otvor dynamicky vyřízne pomocí logiky Geometry Nodes (Mesh Boolean). Při jakékoliv změně rozměrů v panelu se otvor okamžitě zaktualizuje. Po dokončení celé dispozice uživatelka spustí finalizační nástroj, který aplikuje všechny modifikátory, zpracuje UV mapy a připraví čistou statickou síť (mesh) pro renderovací pipeline.

1.4.6 UC 2.3: Rychlá editace vlastností prvků přes kontextovou nabídku

Vizualizátorka potřebuje přiřadit různé materiály podlah jednotlivým místnostem, ideálně bez neustálého přepínání do postranních panelů. Klikne proto pravým tlačítkem myši na plochu místnosti ve viewportu. Vyvolaná kontextová nabídka zobrazí dostupné akce pro daný prvek. Uživatelka zvolí možnost „Změnit materiál podlahy“ a vybere odpovídající texturu. Stejným způsobem může místnost rovnou přejmenovat.

1.4.7 UC 3.1: Rychlý level blackout

Game designér aktivuje kreslicí nástroj a hrubě načrtne sérii navazujících místností. Soustředí se především na proporce a měřítko prostoru vůči hráčské postavě. Addon mezitím průběžně generuje stěny a detekuje vznikající místnosti. Uniformní výšku stěn pro celý půdorys designér následně nastaví v N-panelu.

1.4.8 UC 3.2: Finalizace a export herní úrovně

Na hotovém blackoutu přidá game designér dveřní otvory zadáním požadovaných parametrů v N-panelu u vybraných stěn. Zkontroluje podlahové plochy místností zobrazené v panelu a spustí finalizační nástroj. Ten aplikuje modifikátory Geometry Nodes, zkonvertuje atributy UV map a připraví statickou geometrii pro bezproblémový export do herního enginu ve formátu FBX nebo glTF.

1.4.9 UC 3.3: Interaktivní úprava rozložení místností

Game designér po herním testování (playtestingu) zjistí, že chodba mezi dvěma arénami je pro pohyb hráče příliš úzká. Vybere proto uzel na okraji chodby a pomocí 3D manipulátoru (gizmo) bod plynule posune v rovině XY. Addon automaticky udržuje planaritu a v reálném čase přepočítává sousední místnosti. Výslednou šířku chodby designér ověří čistě vizuálně ve viewportu, aniž by musel zadávat přesné číselné hodnoty.

1.5 Analýza požadavků

Z definovaných person a scénářů použití vyplývá, že modul musí umožňovat interaktivní kreslení půdorysu, parametrickou úpravu stěn a otvorů, automatickou detekci místností i přípravu modelu pro další zpracování (rendering či nasazení v herním enginu). Ne všechny tyto funkce jsou však pro jednotlivé cílové skupiny stejně důležité. Tato sekce proto strukturovaně rozděluje zjištěné potřeby do sedmi funkčních a tří nefunkčních požadavků a ústí v prioritizační analýzu, která hodnotí váhu každého požadavku napříč cílovými skupinami.

1.5.1 Funkční požadavky

Následujících sedm funkčních požadavků pokrývá celý zamýšlený rozsah modulu – od interaktivního kreslení přes parametrickou správu prvků až po finalizaci modelu a integraci pomocných grafických vrstev.

1.5.1.1 FP1 — Interaktivní tvorba místností a kreslení (Pencil Tool)

Nástroj pro kreslení („tužka“) představuje primární vstupní metodu addonu. Umožňuje uživateli definovat půdorys klikáním bodů přímo ve 3D scéně. Jádrem je modální operátor, který po dobu kreslení přebírá veškerou interakci s myší a klávesnicí a průběžně generuje stěny.

Nezbytným minimem pro realizaci (must-have) je kreslení půdorysu v pohledu shora a spolehlivá správa uživatelských vstupů modálním operátorem. Důležitým rozšířením (should-have) je automatické přichytávání (snapping) k osám XYZ odvozené od vzdálenosti kurzoru k existujícím bodům či osám. Jako volitelné vylepšení (nice-to-have) se nabízí průběžné vykreslování náhledu budoucí stěny před jejím potvrzením – systém by neustále sledoval pozici kurzoru, kreslil vodící linku a čekal na kliknutí nebo stisk klávesy Enter.

1.5.1.2 FP2 — Generování a úprava parametrických objektů

Tento požadavek definuje parametrické chování všech prvků půdorysu, tedy stěn i otvorů. Každý objekt si uchovává své parametry (délku, výšku, tloušťku, pozici v prostoru). Při jejich změně se automaticky přepočítá geometrie prvku a dynamicky se zaktualizuje poloha případných navázaných otvorů.

Základní implementace (must-have) vyžaduje dynamickou reprezentaci stěn formou parametrického systému (nikoliv jako statickou síť), okamžitou aktualizaci geometrie při úpravě hodnot, zachování relativní pozice otvorů vůči stěně pomocí pevných datových vazeb a inteligentní generování ořezů (Boolean operací) přímo prostřednictvím Geometry Nodes.

1.5.1.3 FP3 — Správa prostoru a metadat

Správce prostoru tvoří sémantickou vrstvu nad obyčejnou 3D geometrií. Modul musí umět automaticky detekovat uzavřené cykly stěn, rozpoznat je jako samostatné místnosti a vypočítat jejich plochu (celý tento blok je klasifikován jako must-have). Jako volitelné rozšíření (nice-to-have) je koncipována hierarchizace prostorů – organizace místností a celých podlaží do přehledných kolekcí, které umožní například hromadné přepínání viditelnosti v projektu.

1.5.1.4 FP4 — Finalizační nástroj

Finalizační nástroj uzavírá nedestruktivní fázi návrhu. Po dokončení úprav převede parametrický systém na čistou, statickou 3D geometrii, která je připravená pro UV mapování, export do herního enginu nebo nasazení v renderovací pipeline. Hlavním požadavkem (must-have) je trvalá aplikace všech procedurálních generátorů a modifikátorů u vybraných objektů scény.

1.5.1.5 FP5 — Kontextová nabídka

Kontextová nabídka zpřístupňuje akce vázané na konkrétní prvek pomocí plovcí uživatelské nabídky zobrazené přímo u kurzoru. Addon by měl využívat metodu vržení paprsku (raycast) k identifikaci cílového objektu a přes moduly **GPU** nebo **BLF** vykreslovat vlastní rozhraní překrývající 3D viewport. Tato interakční zkratka je hodnocena jako důležité rozšíření (should-have).

1.5.1.6 FP6 — Interaktivní 3D manipulátory

Interaktivní 3D manipulátory (gizma) nabízejí alternativu k ručnímu vypisování parametrů. Umožňují geometrickou manipulaci přímo v prostoru: uživatel uchopí barevné grafické táhlo u daného prvku a tažením myši plynule mění jeho rozměry nebo výšku. Implementace by měla využít nativní rozhraní `bpy.types.Gizmo` a `GizmoGroup`. Funkcionalita je zařazena jako should-have.

1.5.1.7 FP7 — Automatické kótování

Kótovací vrstva průběžně zobrazuje délky stěn a plochy místností jako dynamický text přímo ve viewportu, takže uživatel nemusí zjišťovat rozměry v postranních panelech. Texty generované modulem BLF přes překreslovací smyčku (draw handler) se musejí aktualizovat v reálném čase při každé změně dispozice. Požadavek je klasifikován jako should-have.

1.5.2 Nefunkční požadavky

1.5.2.1 NP1 — Architektura a technologie

Výpočetní jádro modulu je postaveno na Geometry Nodes: logika generování a tvarování geometrie probíhá uvnitř uzlových stromů, zatímco Python plní roli správce, který tyto stromy propojuje a dynamicky upravuje jejich vstupy. Toto striktní oddělení vizuální a aplikační logiky je klíčovým architektonickým principem. Z hlediska distribuce nesmí modul vyžadovat ruční doinstalaci externích knihoven. Cílová platforma (Blender 4.2+) umožňuje definovat závislosti přímo v konfiguračním souboru `blender_manifest.toml`. Externí knihovny, jako například NetworkX využívaná pro grafové výpočty (detekce cyklů, planární embedding), tak budou zabaleny jako standardní Wheel soubory (`.whl`) a nainstalují se automaticky při aktivaci addonu.

1.5.2.2 NP2 — Výkon a nedestructivní přístup

Systém musí reagovat naprosto plynule: uživatel si musí zachovat možnost interaktivní úpravy i při komplexnějších změnách půdorysu a souběžném překreslování geometrie. Hlavním architektonickým důrazem je zde minimalizace výpočetní náročnosti, optimalizace přepočtů parametrů a důsledné respektování grafu závislostí v Blenderu (DepsGraph), aby nedocházelo k neefektivním cyklickým přepočtům celé 3D scény.

1.5.2.3 NP3 — Použitelnost a UX (Uživatelská zkušenost)

Uživatelské rozhraní modulu musí působit jako nativní součást Blenderu. Toho bude dosaženo konzistentním využíváním zabudovaných `UI` komponent (např. `UILayout.row`) a logickým seskupováním nástrojů do přehledných záložek s vysvětlujícími popisky (tooltips). Důraz je kladen na ošetření chyb a srozumitelnou zpětnou vazbu při pokusu o neplatnou operaci – např. při snaze vložit velké okno do příliš krátké stěny.

1.5.3 Prioritizace požadavků

Následující tabulka mapuje každý funkční balíček na scénáře použití, které ho motivují.

Po- ža- da- vek	UC 1.1	UC 1.2	UC 2.1	UC 2.2	UC 3.1	UC 3.2	UC 1.3	UC 2.3	UC 3.3
FP1		●	●		●				
FP2	●	●	●	●	●	●			
FP3	●	●	●		●	●			
FP4				●		●			
FP5								●	
FP6									●
FP7							●		

■ **Tabulka 1** Mapování funkčních balíčků na scénáře použití (● = relevantní)

Každý požadavek je hodnocen zvlášť každou cílovou skupinou (Vysoká / Střední / Nízká / Irelevantní) a výsledná priorita se určuje jako vážený průměr s koeficienty: architekti $\times 3$, vizualizátoři $\times 2$, game designéři $\times 1$. Výsledný průměr $\geq 2,50$ odpovídá prioritě Vysoká (must-have), rozmezí 1,00–2,49 prioritě Střední (should-have).

Poža- davek	Architekti	Vizualizátoři	Game Des.	Průměr	Priorita
FP1 — Kresle- ní	Vysoká	Vysoká	Vysoká	3,00	Vysoká
FP2 — Para- metry	Vysoká	Vysoká	Vysoká	3,00	Vysoká
NP1 — Archi- tektura	Vysoká	Vysoká	Vysoká	3,00	Vysoká
NP2 — Výkon	Vysoká	Vysoká	Vysoká	3,00	Vysoká
NP3 — UX	Vysoká	Vysoká	Vysoká	3,00	Vysoká
FP6 — Mani- puláto- ry	Střední	Střední	Nízká	1,83	Střední
FP3 — Prosto- ry	Vysoká	Nízká	Irelevantní	1,83	Střední
FP7 — Kóto- vání	Vysoká	Nízká	Irelevantní	1,83	Střední
FP4 — Finali- zace	Nízká	Střední	Vysoká	1,67	Střední
FP5 — Kon- text. nabíd- ka	Střední	Nízká	Nízká	1,50	Střední

■ **Tabulka 2** Vážená prioritizace požadavků podle cílových skupin

1.6 Technická analýza

Funkční požadavky říkají *co* má addon umět; technická analýza odpovídá na otázku *jak* — které části Blender API to umožňují, jaké jsou jejich limity a kde hrozí designová nedostatky, které by ovlivnily celou architekturu. Klíčové otázky jsou, jak zachytit kreslení půdorysu v reálném čase bez ztráty výkonu, jak reprezentovat půdorys jako responsivní datový model schopný detekovat místnosti a reagovat na každou změnu stěny, a jak parametrické objekty převést do statické geometrie připravené pro export.

1.6.1 Architektura Blenderu

Blender je modulární systém postavený na unikátním způsobu správy dat — dualitě systému `DNA` a `RNA`. DNA (Blender's Data Architecture) definuje struktury C pro veškerá interní data scény, zatímco RNA (Runtime Notification Architecture) poskytuje reflexivní API, přes které Python přistupuje k těmto strukturám a reaguje na jejich změny [11].

Blender využívá kombinaci vzoru `MVC` (Model-View-Controller), která umožňuje oddělit uživatelské rozhraní (View) od vnitřní logiky (Model) a zpracování vstupů (Controller). Addony mohou definovat vlastní výpočty například v Geometry Nodes (Model), zatímco Blender se stará o jejich vykreslení do viewportu a zachytávání událostí myši přes Python API.

1.6.2 Interaktivní kreslení a interakce ve viewportu

Kreslení stěny musí být plynulé: kurzor se pohybuje, náhledová linka sleduje jeho polohu, stěna se potvrdí kliknutím — ale každý z těchto kroků musí být zpracován v rámci jednoho snímku. Tato sekce vysvětluje, jak Blender takovou interakci umožňuje přes modální operátory a stavový automat, a kde leží výkonnostní limit Pythonu, který je nutné obejít delegováním práce na C++ jádro.

1.6.2.1 Modální operátory

Interaktivní kreslení ve viewportu stojí na modálních operátorech — podtřídách `bpy.types.Operator` [12], které po spuštění zůstávají aktivní a naslouchají událostem myši a klávesnice. Na rozdíl od standardních operátorů, které vykonávají jednorázovou funkci a okamžitě skončí, modální operátor kontinuálně naslouchá událostem generovaným uživatelem nebo systémem.

Je nezbytný pro plynulé kreslení, kdy systém musí v každém okamžiku znát polohu kurzoru a dynamicky na ni reagovat.

Inicializace operátoru začíná metodou `invoke()`, která připraví počáteční stav a registruje operátor do modálního handleru správce oken pomocí `context.window_manager.modal_handler_add(self)`. Od tohoto momentu je každá událost ve viewportu předávána metodě `modal()`. Návrátové hodnoty `modal()` určují, jak Blender s událostí dále naloží:

Hodnota	Funkční dopad	Architektonický význam
<code>RUNNING_MODAL</code>	Operátor pokračuje v běhu	Plynulé tažení stěny nebo změna rozměru
<code>PASS_THROUGH</code>	Operátor běží, událost předána dál	Zoomování a rotace pohledu během kreslení
<code>FINISHED</code>	Operátor končí, generuje Undo krok	Finalizace a uložení stěny do databáze
<code>CANCELLED</code>	Operátor končí bez uložení	Přerušení akce klávesou ESC

■ **Tabulka 3** Návrátové hodnoty modálního operátoru

Metoda `modal()` funguje na principu stavového automatu, který umožňuje operátoru měnit chování v závislosti na fázi interakce. Pro kreslení půdorysu jsou typické stavy: **START** (čekání na první kliknutí), **DRAWING** (průběžné přepočítávání délky a úhlu stěny a vykreslování náhledové linky přes GPU modul), **EXTRUDING** (definování tloušťky nebo výšky) a **FINISHING** (zápis geometrie do scény a čištění draw handlerů).

Stavový automat přináší tři klíčové výhody: řízení složitosti při implementaci víceukrokových nástrojů, možnost kontextového snappingu (v různých stavech jsou aktivní různé typy přichytávání) a optimalizaci výkonu — složité operace se spouštějí pouze při přechodu mezi stavy, zatímco při pohybu myši se aktualizuje pouze drobná vizualizace.

1.6.2.2 Limity výkonu Pythonu v Blenderu

Python je v prostředí Blenderu interpretovaným jazykem, proto je potřeba náročné operace delegovat na stranu Blenderu nebo GPU využívající C++. Zkušenosti z vývoje komplexních generativních nástrojů ukazují, že čistý Python je při hromadném zpracování dat řádově pomalejší. V kontextu architektonického kreslení jsou hlavními limitujícími faktory iterace přes mesh data pomocí `for` smyčky, rostoucí počet unikátních objektů ve scéně a časté aktualizace `DepsGraphu`.

K překonání těchto limitů se v profesionálních addonech využívají tři techniky. Metody `foreach_set` a `foreach_get` umožňují přenášet celá pole dat mezi Pythonem a C++ strukturami Blenderu v jedné operaci namísto nastavování každého vrcholu zvlášť. Delegování na modifikátory spočívá v tom, že Python vytvoří pouze základní čárový model a na něj aplikuje modifikátory jako Solidify nebo Bevel — ty jsou implementovány v C++, plně využívají multithreading a jsou optimalizovány pro real-time aktualizaci. Geometry Nodes jako výpočetní backend pak umožňují Pythonu pouze manipulovat se vstupními hodnotami uzlového stromu, zatímco veškerý výpočet geometrie probíhá v nativním kódu Blenderu.

1.6.3 Reprezentace geometrie

Stěna musí mít přesnou tloušťku v ostrých rozích, reagovat real-time na posun parametru a zároveň generovat čistou topologii pro UV mapování — a ne každý přístup k reprezentaci geometrie tyto tři požadavky splňuje najednou. Tato sekce srovnává dvě dostupné možnosti: imperativní BMesh, který nabízí maximální topologickou kontrolu za cenu výkonu, a deklarativní Geometry Nodes, které výpočet přesouvají do nativního C++ jádra Blenderu.

Geometrie (poloha prvků v prostoru) a topologie (vzájemné vztahy a propojení) tvoří základní dualitu jakékoli 3D struktury. V Blenderu je základní jednotkou mesh složená z vrcholů, hran a ploch.

1.6.3.1 Datová struktura BMesh

BMesh je interní datová struktura Blenderu, která na rozdíl od tradičních struktur založených na trojúhelnících podporuje n-gony (polygony s více než čtyřmi vrcholy). Využívá systém podobný half-edge datovým strukturám, kde jsou vztahy mezi plochami a hranami uloženy tak, aby umožňovaly rychlou navigaci po povrchu sítě.

Z pohledu parametrického modelování nabízí BMesh skrze Python API (modul `bmesh`) nízkoúrovňový přístup k topologii — možnost dotazovat se, které hrany jsou spojeny s daným vrcholem, a tím provádět operace jako `disolve` bez poškození okolní topologie. Algoritmus pro generování stěn obvykle začíná načtením 2D hran, identifikuje uzavřené smyčky jako obrysy místností a operací tloušťky (`offset`) — například přes `bmesh.ops.bevel` nebo posunem vrcholů podél normál hran — vytváří 3D stěny. Tento proces je v Pythonu relativně pomalý, zejména při průběžné validaci integrity sítě.

1.6.3.2 Geometry Nodes a koncept polí

Geometry Nodes (GN) zastupují deklarativní, paralelní přístup: uživatel definuje systém pravidel aplikovaných na celou geometrii současně. Data jsou

reprezentována jako pole atributů vázaných na různé domény (vrcholy, hrany, plochy, instance), přičemž výpočet probíhá v nativním kódu C++ s plným multithreadingem.

Pro generování stěn se v GN nejčastěji používá uzel `Curve to Mesh`. Klíčovou výzvou je Miter Joint problém — standardní vytažení profilu podél křivky vede ke ztenčení stěny v ostrých rozích. Řešením je matematická korekce měřítka profilu v každém bodě křivky pomocí faktoru $f = \frac{1}{\sin(\frac{\theta}{2})}$, kde θ je úhel mezi sousedními segmenty stěny. Tento výpočet se v GN realizuje pomocí vektorové matematiky (skalární součin pro výpočet úhlu) a ač je komplexnější na přípravu, umožňuje dynamicky měnit tloušťku stěn pouhým posunutím bodu v 2D půdorysu.

Charakteristika	BMesh	Geometry Nodes
Způsob práce	Iterativní / Imperativní	Paralelní / Deklarativní
Výkon	Omezený interpretací Pythonu	Vysoce optimalizované C++
Topologická flexibilita	Absolutní	Omezená na definované uzly
Vizuální odezva	Po spuštění skriptu	Real-time ve view-portu
Vytváření tloušťky	Přesné, výpočetně drahé	Vyžaduje manuální korekci
Multithreading	Ne (omezení GILu)	Ano (nativní)
Stabilita topologie	Riziko non-manifold chyb	Stabilnější

■ **Tabulka 4** Srovnání BMesh a Geometry Nodes pro generování 3D stěn

1.6.4 Datový model

Samotná 3D síť (mesh) sice přesně zachycuje tvar a polohu stěn v prostoru, ale nenese žádné sémantické informace o prostorech samotných — nemá jak zjistit, zda spolu dvě místnosti sousedí, jaký mají účel nebo jaké jsou jejich parametry. Parametrický modul, jehož úkolem je automaticky detekovat místnosti, počítat jejich podlahové plochy a dynamicky reagovat na každou topologickou změnu, proto nevyhnutelně vyžaduje dedikovanou datovou vrstvu. Tato sekce analyzuje možné přístupy k její reprezentaci.

1.6.4.1 Reprezentace bez samostatné datové vrstvy

Nejjednodušší možný přístup vůbec nevytváří oddělenou datovou strukturu: půdorys je uložen výhradně v geometrii Blenderu a veškerá sémantická data jsou vázána přímo na konkrétní prvky sítě pomocí uživatelských vlastností (Custom Properties) nebo pojmenovaných atributů (Named Attributes). V takovém modelu představují hrany sítě stěny a jednotlivé polygony (faces) tvoří místnosti.

Hlavní výhodou je jednoduchost — řešení nevyžaduje žádné externí knihovny a nabízí přímou kompatibilitu s modifikátory Geometry Nodes, které umějí pojmenované atributy číst v reálném čase. Zásadní nevýhodou je však velmi nízká efektivita dotazování. Hledání odpovědi na otázky typu „sousedí místnost A s místností B?“ nebo „kolik místností je dostupných z hlavní haly?“ vyžaduje iterativní procházení celé topologie sítě s časovou složitostí $O(E)$ pro každý jednotlivý dotaz.

1.6.4.2 Lineární datové struktury

Druhý přístup udržuje aplikační stav v Pythonu pomocí plochých (flat) seznamů nebo slovníků. Eviduje uzly (styky stěn), stěny a místnosti jako samostatné objekty. Každý prvek má přiřazen jednoznačný identifikátor (ID) a informace o sousednosti je uložena přímo v záznamu daného prvku jako seznam ID jeho sousedů.

Tato varianta je plně implementovatelná pomocí standardních knihoven Pythonu a lze ji snadno testovat i mimo prostředí Blenderu. Jejím hlavním limitem je však rychlé snížení výkonu při složitějších topologických operacích. Například automatická detekce nově vzniklých místností (uzavřených cyklů) by vyžadovala implementaci vlastních algoritmů pro prohledávání grafu a výpočetní náročnost při opakovaném přepočítávání sousednosti by exponenciálně rostla s každým novým prvkem.

1.6.4.3 Grafová reprezentace

Z matematického hlediska je nejpřirozenějším modelem půdorysu neorientovaný planární graf [13], kde uzly (V) odpovídají stykům stěn a hrany (E) představují osy samotných stěn. Tento formát otevírá cestu k využití standardních, vysoce optimalizovaných grafových algoritmů: od detekce minimálních cyklů (místností), přes ověřování planarity, až po výpočty dostupnosti. Pro Python navíc existuje robustní knihovna NetworkX [14], která všechny tyto operace nativně implementuje a umožňuje jejich testování zcela nezávisle na Blenderu.

Hlavním architektonickým rozhodnutím při použití grafové reprezentace je hloubka struktury modelu: tedy zda zachovat jeden sjednocený graf kombi-

nující topologii se sémantikou (uzly nesou jak geometrické souřadnice, tak metadata místnosti), nebo striktně oddělit topologický skelet (uzly = spoje stěn, hrany = osy stěn) od sémantického grafu (uzly = místnosti, hrany = sousedství). Sjedený graf je jednodušší na průběžnou správu. Oddělené vrstvy sice vyžadují synchronizaci, ale mnohem lépe izolují zodpovědnosti jednotlivých domén. Rozdíl ve výkonu je zde markantní: zatímco v čistě topologickém grafu má zjištění sousednosti dvou místností složitost $O(E)$, v explicitním sémantickém grafu jde o triviální dotaz se složitostí $O(1)$.

S grafovou reprezentací úzce souvisí i strategie automatické detekce místností při uzavírání stěnových cyklů, jíž se detailně věnuje následující podkapitola.

1.6.4.4 Způsoby detekce místností

Pro detekci místností existují dva přístupy. *Eager* přístup ihned při vzniku stěny vytváří dočasný objekt místnosti, který je při uzavření cyklu validován. Toto vede k řadě problémů: nedefinovanému stavu dočasného objektu, konfliktu při slučování po uzavření cyklu (systém musí vybrat, který dočasný objekt zachovat), problémům se simultánním uzavřením více cyklů a složité vlastní správou Undo historie.

Lazy přístup naproti tomu vytváří místnost výhradně tehdy, kdy je detekován uzavřený cyklus v strukturálním grafu. Invariant `Room` $\leftrightarrow$ minimální uzavřený cyklus platí vždy a plně — neexistuje žádný stav „rozpracované místnosti“. `NetworkX` vrátí seznam všech nových minimálních cyklů najednou, pro každý vznikne jeden `Room` node bez spojovací logiky. Undo je přirozené: odebrání uzavírající stěny znamená zánik cyklu a zánik `Room` nodu. *Lazy* detekce zajišťuje determinismus — stejná topologie Vrstvy 1 vždy produkuje stejnou Vrstvu 2 bez závislosti na pořadí editací.

1.6.5 Tvorba otvorů pro okna a dveře

Okno osazené ve stěně se musí pohybovat spolu s ní, přizpůsobit při změně její tloušťky a okamžitě regenerovat otvor bez jakéhokoli ručního zásahu — to je základní podmínka nedestruktivního workflow. Splnit ji lze více způsoby, přičemž každý nabízí jiný kompromis mezi výkonem, numerickou stabilitou a topologickou čistotou výsledné geometrie. Tato sekce porovnává následující tři hlavní přístupy jak zmíněnou podmínku splnit:

Boolean operace spravované přes Python API využívají standardní modifikátorový stack, kde Python dynamicky vytváří cutter objekty a přiřazuje je ke stěně. Hlavní nevýhodou je vysoká režie při správě stacku s desítkami modifikátorů a numerická nestabilita — pokud souřadnice po transformaci

nejdou binárně identické (kvůli zaokrouhlení floatů), **Exact solver** může selhat při detekci společných ploch.

Mesh Boolean v Geometry Nodes nahrazuje objektový stack jedním uzlovým stromem, kde operace probíhají nad toky geometrických dat. Na rozdíl od modifikátorů, které pracují v párech, uzel **Mesh Boolean** v GN dokáže zpracovat celé kolekce instancí oken jako jeden sloučený vstup — to dramaticky snižuje počet reevaluačí. Problémem je, že solver považuje vše zapojené do vstupu za jediné těleso; pokud se dvě stěny překrývají, může dojít ke vzniku dutých výsledků.

Metody bez Boolean operací se vyhýbají výpočtům průsečíků a otvory vkládají přímo do procesu generování topologie. **Curve Trimming** ořízne vodící křivku před vytažením do 3D, čímž vzniknou fyzické mezery s čistou topologií — tato metoda však vyžaduje reprezentaci stěn jako **Blender Curve** objektů a není kompatibilní s architekturou pracující s **base meshem** a **Named Attributes**. Modulární instancování chápe stěnu jako pole buněk s plnými nebo prázdnými moduly. **Vertex Group Topology** označí specifické vrcholy atributem **is_window** a GN je posunou aby vytvořily pravoúhlý otvor.

Následující tabulka zobrazuje hlavní rozdíly mezi zmíněnými přístupy:

Parametr	API Modifikátor	GN Mesh Boolean	Curve Trimming
Výpočetní složitost	$O(n \times m)$	$O(m \log m)$	$O(n)$
Numerická stabilita	Nízká (float drift)	Střední	Absolutní
Topologický výstup	Artefakty, n-gony	n-gony, nutný cleanup	Perfektní quads
Flexibilita	Vysoká	Vysoká	Omezená

■ **Tabulka 5** Srovnání metod pro tvorbu otvorů ve stěnách

1.6.6 Ukládání dat a správa metadat

Parametrický addon pracuje s daty na dvou odlišných úrovních: s geometrií stěn, která musí přežít každý zpětný (Undo) krok, i s metadaty místností — jejich názvem, typem a plochou — která musí zůstat konzistentní při sdílení souborů nebo instancování objektů. Tato sekce analyzuje, které mechanismy **Blender API** pro tato data nabízí, na které úrovni hierarchie **BLenderu** je přirozeně ukládat a jak zajistit, aby grafové struktury v paměti přežily uložení a opětovné otevření **.blend** souboru.

1.6.6.1 Systémy pro správu uživatelských parametrů

Blender nabízí dva hlavní mechanismy. **Vlastnosti ID** (Custom Properties) jsou flexibilní mechanismus pro připojování libovolných dat k jakémukoliv datovému bloku — ukládají celá čísla, desetinná čísla, řetězce a pole. Pro komplexní architektonické systémy mají nedostatky: absenci striktní typové kontroly a omezené možnosti definice logiky při změně hodnoty. **Modul bpy.props** umožňuje definovat vlastnosti registrované přímo do systému RNA s plnou podporou zpětných volání (`update`, `get`, `set`), klíčových pro reaktivní architektonické prvky.

Pro správu informací o prostorech (název, plocha, typ místnosti, výška) je optimálním vzorem `bpy.types.PropertyGroup`, který seskupuje související parametry do logického celku připojeného k datovému bloku pomocí `PointerProperty` (vazba 1:1) nebo `CollectionProperty` (vazba 1:N).

1.6.6.2 Vazby dat na úrovně hierarchie Blenderu

Rozhodnutí, na jakou úroveň hierarchie Blenderu budou metadata uložena, má hluboké důsledky pro stabilitu addonu a chování při Undo/Redo.

Úroveň **Scéna** (`bpy.types.Scene`) je vhodná pro globální parametry projektu. Data specifická pro jednotlivé místnosti jsou na této úrovni nevhodná: smazání objektu ve viewportu nezpůsobí automatické smazání metadat v kolekci, což vede k hromadění dat. Úroveň **Objekt** (`bpy.types.Object`) nese informace o transformaci a viditelnosti. Komplikace nastávají při instancování: vytvoření instance přes Alt+D vytvoří dva objekty sdílející stejná geometrická data, ale s unikátními daty na úrovni Object — změna rozměru jednoho okna by se neprojevila u ostatních instancí. Úroveň **Geometrie** (`bpy.types.Mesh`) je nejvhodnějším přístupem pro architektonické prvky: mesh reprezentuje „definici typu“, prvku a všechny sdílené instance mají identické parametry, v souladu s principy BIM.

1.6.6.3 Perzistence grafových dat

Blender při uložení `.blend` souboru automaticky ukládá mesh geometrii a Custom Properties objektů — Python objekty v paměti (např. NetworkX grafy) nikoli. Po zavření a opětovném otevření souboru jsou grafy ztraceny, pokud nejsou zachovány.

Přístup	Princip	Hlavní nevýhoda
JSON v Custom Property	Serializovat grafy do JSON stringu	Redundance; nutno verzovat schéma
Pickle v Custom Property	Serializovat Python objekty do bajtů	Bezpečnostní riziko; citlivost na verze
Rekonstrukce z meshe	Po načtení přebudovat grafy z uložené topologie	Mesh musí být jediným zdrojem pravdy

■ **Tabulka 6** Přístupy k perzistenci grafových dat v Blenderu

1.6.6.4 Perzistence globálních nastavení addonu

Addon spravuje dvě kategorie nastavení. **Projektová data** (výchozí tloušťka stěny, výška, systém jednotek) přímo ovlivňují geometrii konkrétního půdorysu. **Nastavení chování addonu** (výkon, cesty k souborům, UI preference) jsou naopak nezávislá na projektu.

Pro každou kategorii nabízí Blender jiný mechanismus: `bpy.types.AddonPreferences` ukládá data globálně do `userpref.blend` (platí napříč projekty), zatímco `bpy.types.PropertyGroup` na `Scene` ukládá data per-projekt do aktuálního `.blend` souboru. Analýza existujících addonů ukazuje, že oba hlavní architektonické addony (Archimesh, Archipack) projektová data do `AddonPreferences` neukládají — ten rezervují výhradně pro chování addonu. Pro `FloorPlanMaster` je proto optimálním řešením **Scene PropertyGroup** se zabezpečenými výchozími hodnotami: projektové hodnoty jsou součástí `.blend` souboru, jsou tedy přenositelné a verzovatelné s projektem. `AddonPreferences` se použijí výhradně pro nastavení nesouvisející s konkrétním projektem.

1.6.7 Uživatelské rozhraní

Architekt, který musí odtrhnout pohled od půdorysu, aby v postranním panelu dohledal tlačítko, ztrácí soustředění právě ve chvíli, kdy je nejcennější. Dobrý UI addon proto nesmí nutit uživatele hledat — parametry stěny se musí zobrazit samy při jejím výběru, délky stěn musí být čitelné přímo ve viewportu a editace hodnotou i tažením myši musí být dostupné ze stejného místa. Tato sekce popisuje, jaké mechanismy Blender API pro takové rozhraní nabízí a jaké UI vzory se v existujících architektonických nástrojích opakují a proč.

UI systém Blenderu je postaven na principu Immediate Mode rendering — rozhraní se kompletně překresluje při každém snímku nebo změně. Logika určující, co se má zobrazit, musí být extrémně rychlá; výhodou je flexibilita — rozhraní vždy dokonale odráží aktuální stav bez synchronizace. Prostor obrazovky je hierarchicky členěn: Screen (celé hlavní okno), Area (obdélníkové oblasti) a Region (specifické části každé oblasti, v 3D Viewportu to jsou hlavní 3D zobrazení, Sidebar, Header a Toolbar). K propojení UI s daty scény slouží RNA systém: data v rozhraní jsou pouze vizuálním ukazatelem do datových struktur C++ jádra a uživatelská změna hodnoty se okamžitě propíše přes RNA přímo do databáze scény.

1.6.7.1 GPU modul a draw handlers

Modul `gpu` slouží jako abstrakční vrstva nad nízkoúrovňovými grafickými API (OpenGL, Metal, Vulkan) a je pro architektonický addon klíčový: umožňuje vykreslovat vodící linky, kóty a náhledy stěn přímo na GPU bez nutnosti vytvářet geometrii v databázi Blenderu. Draw handlers se registrují k `bpy.types.SpaceView3D` metodou `draw_handler_add`. Existují dva hlavní režimy: `POST_VIEW` pracuje v souřadném systému 3D scény (ideální pro vodící linky) a `POST_PIXEL` pracuje v souřadnicích obrazovky a je nezbytný pro textové popisky a kóty, které mají zůstat čitelné nezávisle na míře přiblížení. Každý přidáný handler musí být při ukončení operátoru odstraněn pomocí `draw_handler_remove()`.

Pro kótovací popisky lze použít modul BLF (Blender Font Library), který vykresluje text přímo do viewportu s možností kontroly nad pozicí a vzhledem. Pro kótovací overlay je rozhodující čitelnost nezávislá na úhlu pohledu — GPU `POST_PIXEL` overlay tuto podmínku splňuje; alternativní přístup s GN String to Curves generuje 3D mesh textu, který je při šikmém pohledu hůře čitelný.

1.6.7.2 UI vzory v architektonických nástrojích

Analýza existujících řešení odhalila opakující se UI vzory, které cílová uživatelská skupina již ovládá. **Sidebar / Properties panel** — parametry vybraného objektu jsou trvale viditelné v postranním panelu; Archipack posouvá tento vzor dál automatickým zobrazením parametrů v N-panelu při výběru objektu bez nutnosti klikat na tlačítko. **Gizmo při výběru (Auto-manipulate on select)** — při výběru stěny se automaticky zobrazí táhla pro tloušťku a výšku, vhodné pro geometrické úpravy tažením myši. **HUD během modálního nástroje** — aktuální hodnoty a nápověda kláves zobrazeny přímo ve viewportu, nezbytné pro nástroje s více stavy. **Kontextová nabídka na RMB** — přístup k méně frekventovaným akcím (přejmenování,

smazání, nastavení materiálu) je primární UI konvencí Blenderu; nabídka je závislá na vybraný prvek, čímž redukuje vizuální šum. **Pop-over dialog** — otevírá se u kurzoru, poskytuje prostor pro textový vstup nebo výběr z enumu a zavře se kliknutím mimo bez nutnosti potvrzení. **Jednoznakové zkratky** — odpovídající prvnímu písmenu akce nebo ustálené konvenci prostředí, snižují latenci opakovaných akcí bez přepnutí pohledu na toolbar.

1.6.8 Finalizační nástroj

Parametrický model funguje jako živý organismus plný vzájemných závislostí: Geometry Nodes v reálném čase přepočítávají tvary geometrie, pojmenované atributy uchovávají data o místnostech a modifikátory čekají na své konečné uplatnění. Renderovací a herní enginy však vyžadují přesný opak – statickou geometrii (mesh) s čistými UV kanály a bez zbytečných materiálů, která už není nijak svázána s původním procedurálním výpočtem. Tato sekce analyzuje, jak takovou konverzi bezpečně provést pomocí vyhodnoceného grafu závislostí (Evaluated Depsgraph), aby nedošlo ke ztrátě dat ani k nevratnému zničení původního modelu.

Nástroj pro finalizaci tvoří pomyslnou tečku za nedestruktivní fází návrhu. Jeho úkolem je převést procedurální geometrii do statické, topologicky čisté a plně optimalizované sítě. Technicky se tento proces opírá o rozdíl mezi původními daty (Original Data, tedy čistou sítí bez modifikátorů) a vyhodnoceným objektem (Evaluated Object), což je stav po aplikaci všech modifikátorů a uzlových stromů. Vzhledem k tomu, že Blender používá pro správu vztahů mezi objekty centrální graf závislostí (DepsGraph), je pro finalizaci klíčové získat právě jeho vyhodnocenou verzi, která reprezentuje finální stav scény.

Abychom získali čistou síť připravenou pro export, je nezbytné načíst vyhodnocený stav objektu pomocí metody `evaluated_get()` a z něj následně vygenerovat novou statickou geometrii voláním `bpy.data.meshes.new_from_object(obj_eval)`. Tento postup zaručuje naprostou nedestruktivnost – původní parametrický objekt zůstává nedotčen a uživatel se k němu může kdykoliv vrátit. Naopak běžný postup, který spoléhá na procházení seznamu `obj.modifiers` a postupné volání `modifier_apply()`, bývá vysoce rizikový. Každá úspěšná aplikace modifikátoru totiž změní indexy těch zbývajících. V praxi se proto osvědčují spolehlivější přístupy: buď iterace přes předem připravený statický seznam názvů modifikátorů, nebo použití cyklu `while` s integrovaným ošetřením chyb, který problematické položky jednoduše přeskočí.

Zvláštní pozornost vyžadují UV mapy a materiály. V prostředí Geometry Nodes jsou UV mapy ukládány pouze jako 2D vektory v rozích polygonů (v doméně Face Corner). Pokud by zůstaly v této podobě, exportéry do formátů

FBX nebo glTF by je zcela ignorovaly. Po aplikaci procedurální geometrie je proto nezbytné tyto pojmenované atributy explicitně převést na standardní UV vrstvy. Další komplikace vzniká při slučování instancí – často totiž dochází ke kumulaci geometrií s odlišnými definicemi, čímž vznikají zbytečně duplicitní materiálové sloty. V herních enginech by tento stav vedl k nežádoucímu nárůstu vykreslovacích požadavků (draw calls). Závěrečná finalizace by proto měla zahrnovat důkladné čištění materiálů, při kterém nástroj zanalyzuje existující sloty, identifikuje duplikáty, pomocí modulu BMesh přemapuje indexy a následně odstraní veškeré prázdné a nadbytečné pozice.

Kapitola 2

Návrh

Kapitola 3

Implementace

Kapitola 4

Testování

Závěr

TODO

Bibliografie

- [1] Blender Foundation, „Blender – Open Source 3D Creation Suite". 2024.
- [2] Architizer, „Architecture 101: What Is Parametric Architecture?". 2023.
- [3] ViBIM Global, „BIM vs CAD: Key Differences and When to Use Each". 2024.
- [4] A. Vazquez, „Archimesh – Architectural Objects for Blender". 2024.
- [5] S. Leger, „Archipack – Architectural Design Addon for Blender". 2024.
- [6] IfcOpenShell Contributors, „BonsaiBIM – Native IFC Authoring Platform for Blender". 2024.
- [7] International Organization for Standardization, „Industry Foundation Classes (IFC) for data sharing in the construction and facility management industries – Part 1: Data schema". 2024.
- [8] Trimble Inc., „SketchUp". 2024.
- [9] Autodesk Inc., „AutoCAD". 2024.
- [10] Autodesk Inc., „Revit". 2024.
- [11] Blender Foundation, „Blender Developer Documentation". 2024.
- [12] Blender Foundation, „Blender Python API Documentation". 2024.
- [13] S. Hu, W. Wu, Y. Wang, B. Xu, a L. Zheng, „GSDiff: Synthesizing Vector Floorplans via Geometry-enhanced Structural Graph Generation", *arXiv preprint arXiv:2408.16258*, 2024.
- [14] A. A. Hagberg, D. A. Schult, a P. J. Swart, „Exploring Network Structure, Dynamics, and Function using NetworkX", in *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, G. Varoquaux, T. Vaught, a J. Millman, Ed., 2008, s. 11–15.

Contents of the attachment

└─ TODO TODO